

Reviewer Pack — Free Entropy Gap of Read-Twice BP Transition Matrices for IP...

Entry c497cb4d0173 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 03:08:44 UTC

Free Entropy Gap of Read-Twice BP Transition Matrices for IP_2

Entry ID: c497cb4d0173 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 03:08:44 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP P computing IP_2 , the free entropy $\chi(P) \geq \Omega(n)$, while for read-once BPs, $\chi(P) \leq O(\log n)$. For random read-twice BPs with n variables, $\chi(P) = \Theta(\sqrt{n} \log n)$ with probability $\geq 2/3$.

Rationale (proposer's reasoning):

Free entropy quantifies non-commutative independence structures. Read-twice BPs exhibit inherent non-commutative correlations in their transition matrices, while read-once BPs can be modeled as commutative structures. This gap mirrors the exponential vs polynomial size separation for IP_2 .

Taxonomy category: BP_READTWICE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): da3e24b77cc2970d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_det(A):
        n = len(A)
        if n == 1:
            return A[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1)**j * A[0][j] * matrix_det(submatrix)
        return det

    def free_entropy(P):
        n = len(P)
        eigenvalues = []
        for i in range(n):
            for j in range(n):
                if i != j:
                    submatrix = [[P[k][l] if (k != i and l != j) else 1 - P[k][l] for l in range(n)]
                    eigenvalues.append(matrix_det(submatrix))
        return sum(math.log(abs(e)) for e in eigenvalues) / n

    def generate_bp(n):
        bp = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if random.choice([True, False]):
                    bp[i][j] = 1
                    bp[j][i] = 1
        return bp

    def is_read_twice(bp):
        n = len(bp)
        for i in range(n):
```

```

        for j in range(i+1, n):
            if bp[i][j] == 0:
                continue
            for k in range(j+1, n):
                if bp[j][k] == 0 and bp[k][i] == 0:
                    return True
        return False

n = 40
P = generate_bp(n)

if not is_read_twice(P):
    return {
        "metric_name": "free_entropy",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

chi_P = free_entropy(P)

if chi_P is None:
    return {
        "metric_name": "free_entropy",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

return {
    "metric_name": "free_entropy",
    "metric_value": chi_P,
    "instances_tested": 1,
    "conjecture_holds": True if chi_P >= math.sqrt(n) * math.log(n) else False,
    "counterexample": "" if chi_P >= math.sqrt(n) * math.log(n) else f"chi(P) = {chi_P}, expected"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    total_metric_value = 0
    count_supporting = 0
    counterexamples = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if trial_result["conjecture_holds"]:
            total_metric_value += trial_result["metric_value"]
            count_supporting += 1
        else:
            counterexamples.append(trial_result["counterexample"])

```

```

mean_metric_value = total_metric_value / len(seeds)
std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in [r["metric_value"] for
support_fraction = count_supporting / len(seeds)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif counterexamples:
    first_counterexample = counterexamples[0]
    print(f"RESULT: FALSIFIED counterexample=\"{first_counterexample}\" first_failing_seed={seeds[counterexamples[0]]}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and memory limits to complete execution

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	45500
2	preregistration	ollama_remote	qwen3:8b	0	0	32037
3	novelty	ollama_remote	qwen3:8b	0	0	27492
4	novelty	ollama_remote	qwen3:8b	0	0	21581
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13609
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9253
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8382
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11268
9	judge	ollama_remote	qwen3:8b	0	0	24711

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193833 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c497cb4d0173.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c497cb4d0173.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c497cb4d0173.tar.gz` (if generated)